



The Upskilling Program ICT Upskilling Program

AlHussein Technical University (HTU)

Quality Assurance and Software Testing

Prepared by: Rania Hasan

February 2026

Acknowledgement

I would like to express my deepest gratitude to everyone who supported me throughout the course of this project. Special thanks to Mr. Ashraf Al-Smadi for his guidance, feedback, and technical mentorship during my training; Mr. Brandon Copeland, our English instructor, for helping me enhance my technical communication and writing skills; and Ms. Hala Abu Rubeiha for her support in developing my soft skills.

This journey has been incredibly educational. I have learned a lot, grown both professionally and personally, and gained the confidence and skills needed to enter the workforce. I am now better prepared to face real-world challenges, tackle complex problems, and contribute effectively in a professional environment.

Finally, I would like to thank Al Hussein Technical University for providing this valuable training opportunity and for creating an environment that nurtures growth, learning, and excellence.

Contents

Executive Summary	4
Introduction	5
Project Background	5
Objectives	5

Expected Outcomes	5
System Under Test (SUT)	6
Web Application Overview	6
Rest API Overview	6
Assumptions and Constraints	7
Test Strategy and Planning	7
Test Scope & Out of Scope	7
Test Type	8
Test Environment.....	8
Test Data Strategy.....	8
Key requirement.....	9
Entry & Exit Criteria	9
Risks and Mitigation Plan.....	9
Manual Testing.....	10
Api Testing.....	10
UI Automation Testing (Selenium + TestNG)	11
Test Scenarios Automated	12
How to Run the Automation Tests	12
Automation Results Process	13
CI/CD and Version Control	13
Git Workflow and Repository Structure	13
Branching and Commit Strategy	13
Benefits of CI/CD Integration	13
Automation Execution via GitHub Actions	14
Risks, Limitations, and Challenges	14
Conclusion	14
Appendix.....	15

Executive Summary

This report summarizes the Quality Assurance activities carried out for the Loyalty+ Store retail platform as part of the ICT Upskilling Program – Software Testing and Quality Assurance track. The main goal of this project was to apply the knowledge and skills gained throughout the course in a practical, real-world testing scenario that covered both functional and non-functional testing.

The system under test consisted of a web-based retail application (SauceDemo) and a RESTful backend API (DummyJSON). The testing process included test planning, manual testing, defect reporting, API testing with Postman, API performance testing with k6, and UI automation testing with Selenium WebDriver and JavaScript.

Manual testing focused on validating essential user journeys such as login, product browsing, cart management, and checkout. During execution, multiple defects and test findings were identified, documented, and prioritized based on their severity and potential impact on the user experience. API testing was designed to ensure correct request and response behavior, proper authentication handling, validation of critical data fields, and coverage of both positive and negative scenarios. Automated API executions were performed using Newman, while performance testing evaluated system behavior under different load conditions, focusing on response time, throughput, and error rate.

Overall, the testing results show that the system performs reliably for its core functionalities under normal usage conditions, while also highlighting areas that require improvement. The findings and recommendations presented in this report aim to enhance system stability, performance, and overall quality. This document serves as a complete and structured QA testing portfolio, supported by clear evidence, automation scripts, and analysis.

Introduction

This section provides an overview of the project background and outlines the main objectives of the testing activities conducted for the Loyalty+ Store platform. It explains the context of the system under test and the purpose of performing comprehensive Quality Assurance testing as part of this capstone assignment.

Project Background

The project represents a realistic retail platform that includes a web-based customer interface and a backend REST API. The platform allows users to perform common e-commerce actions such as logging in, browsing products, adding items to a cart, and completing purchases. Due to the critical nature of these features, ensuring system quality, reliability, and performance is essential for providing a smooth and trustworthy user experience.

Objectives

The primary objective of this testing engagement was to evaluate the quality of the platform through structured and systematic testing activities. The specific objectives of this project were to:

- Verify that the core functional requirements of the web application work as expected.
- Identify, document, and prioritize defects based on severity and business impact.
- Validate REST API behavior through functional, negative, and data-driven testing using Postman.
- Assess API performance under different load conditions using k6.
- Automate critical UI test scenarios using Selenium WebDriver with JavaScript.
- Ensure that all tests are repeatable, well-documented, and supported by clear execution evidence.

Expected Outcomes

By the end of this project, the following outcomes are expected:

- A complete manual test plan and test cases for the web application
- Automated UI test scripts
- API functional testing with automated assertions
- API performance testing with smoke and load profiles

- Execution reports and performance metrics
- A structured QA report documenting all testing activities

System Under Test (SUT)

This section describes the systems selected for testing in this project. The System Under Test (SUT) includes both a web-based user interface and a backend REST API, representing a complete retail platform where frontend and backend components work together to deliver core e-commerce functionality.

Web Application Overview

The web application under test is SauceDemo, a publicly available retail demo platform that simulates an online shopping experience. The application allows users to authenticate, browse a product catalog, add items to a shopping cart, and complete the checkout process. It also provides access to order-related information, making it suitable for validating critical user flows commonly found in real-world retail systems.

From a testing perspective, SauceDemo offers a stable environment to assess UI behavior, usability, and functional correctness. Its dynamic elements and end-to-end user journeys make it an effective platform for manual testing as well as UI automation, enabling validation of both positive and negative scenarios.

Rest API Overview

The backend system under test is DummyJSON, a RESTful API designed for testing and prototyping purposes. It provides a variety of retail-like resources, including authentication, products, carts, and users, exposed through standard HTTP methods. The API supports both public and protected endpoints, allowing authentication and authorization flows to be tested.

DummyJSON was used to validate API functionality, request and response structures, error handling, and data consistency. Although write operations are simulated and may not persist changes, the API reliably reflects realistic backend behavior, making it suitable for functional, negative, and performance testing scenarios.

Assumptions and Constraints

The following assumptions and constraints were considered during the testing process:

- All testing was performed using test data only, with no real personal or payment information.
- The systems under test are demo platforms and may change without prior notice.
- API write operations are simulated and do not guarantee data persistence.
- Security testing was intentionally excluded, as per project guidelines.
- Performance testing was conducted within safe and respectful load limits to avoid service disruption.

Test Strategy and Planning

A solid test strategy ensures the Loyalty+ Store works reliably, delivers a seamless user experience, and targets the most critical issues for actionable insights.

Test Scope & Out of Scope

The **testing scope** focused on the features and scenarios that directly impact the end-user experience and business functionality.

This included:

- Functional testing of key web flows: login, product browsing, cart operations, checkout, and order history.
- API testing covering authentication, product and cart management, and user-related operations.
- UI automation to validate repeatable, high-risk flows.
- Performance testing for selected API endpoints to assess responsiveness under load.

Out-of-scope areas were defined to keep efforts realistic and targeted:

- Security testing (e.g., penetration or vulnerability scans)
- Third-party services such as payment gateways or email notifications.
- Non-critical UI elements, including visual styling like colors and fonts.

Test Type

To provide a complete quality assessment, multiple types of testing were applied:

- Manual Functional Testing: To explore behavior, detect logic errors, and validate user journeys.
- Negative Testing: To ensure the system handles invalid inputs and edge cases gracefully.
- Automated Testing: For repeatable, high-risk scenarios to increase reliability and efficiency.
- API Performance Testing: To measure response time, throughput, and error rates under various load conditions.
- Exploratory Testing: To uncover unexpected behaviors and edge cases beyond predefined test cases.

Test Environment

Testing was conducted in controlled environments to ensure consistency and reliability. The setup included:

- Web: Edge browser on a Windows laptop to access the SauceDemo UI.
- API: Postman for manual requests, Newman for automated collection runs, and k6 for performance testing.
- Automation: Selenium WebDriver with JavaScript, executed via IDEs on the same laptop.
- Data: Only test data was used, with no real personal or payment information.

Test Data Strategy

The test data was designed to reflect real user behavior and cover many scenarios:

- User Accounts: Fake users and addresses were created. In automation, random users and inputs were generated to make tests more dynamic.
- Products and Cart: Different product selections and cart quantities were used to test normal and boundary cases.
- Negative and Edge Cases: Invalid credentials, empty carts, and out-of-range product IDs were tested to check system handling.
- Boundary Testing: Values at the limits of acceptable ranges were used to find potential issues.

Key requirement

- Only logged-in users can proceed to checkout
- Users must be able to add, update, and remove products from the cart
- Cart totals must update correctly when quantities change
- Checkout flow must complete successfully without errors
- The confirmation page must display a valid order confirmation message

Entry & Exit Criteria

Entry Criteria:

- Test environments set up and accessible.
- All necessary test accounts and data prepared.
- Postman collections and automation scripts ready.

Exit Criteria:

- All planned manual and automated tests executed.
- Defects logged, prioritized, and verified.
- API performance results captured and analyzed.
- Summary reports and evidence compiled.

Risks and Mitigation Plan

Risk	Mitigation
Dependency on external demo platforms	Documented any outages and used fallback endpoints when necessary
Non-persistent data in DummyJSON	Designed tests to validate responses rather than rely on stored state
Performance test limits	Controlled load carefully to avoid service disruption
Human errors in manual testing	Used checklists, peer reviews, and captured screenshots for accuracy

Manual Testing

Manual testing was performed to validate the core functionality of the Loyalty+ Store platform. The goal was to ensure that critical user flows work as expected, to detect unexpected behavior, and to provide clear evidence of any issues for further improvement.

For more detailed information, including the full test cases, execution evidence, screenshots, bug report, and the complete test plan, please refer to the provided links.

Test plan	https://docs.google.com/document/d/1hnrHvD0MdYAzM0lw_gLEn04fwwdGmEAE/edit?usp=drive_link&oid=113345083993190881514&rtpof=true&sd=true
Test cases	https://docs.google.com/document/d/1ETyhPYnsdmlisSEZ5Xg3teMcy9b1Qj22/edit?usp=drive_link&oid=113345083993190881514&rtpof=true&sd=true

Api Testing

API testing was a central part of this QA project, ensuring that the backend services of the DummyJSON behave correctly, return accurate data, and handle both valid and invalid inputs gracefully. Testing focused on critical endpoints for authentication, products, carts, and users. During testing, a bug was discovered, which was carefully documented and reported with clear reproduction steps, severity, and evidence. This allowed proper tracking and resolution of the issue.

The testing included:

- Postman Collection Design: Organized requests covering REST operations (GET, POST, PUT, DELETE) with clear structure and naming.
- Environment Variables and Authentication Handling: Used variables for base URLs, tokens, and IDs to make the collection reusable and maintainable.
- Automated Assertions and Test Scripts: Verified status codes, response fields, and error messages for accuracy and reliability.

- Data-Driven Testing: Executed tests using multiple inputs to cover various scenarios, including negative and boundary cases.
- Newman Execution and Report Summary: Automated collection runs generated an HTML report for clear visibility of results.

For more detailed information, including the full Postman collection, test scripts, environment setup, execution reports, and the documented bug report, please refer to the provided links.

K6	https://drive.google.com/drive/folders/1dfVc4Jx_RU03-Mz0gJOxo4R9BTEkWdQr?usp=drive_link
Newman+Html report	https://drive.google.com/drive/folders/1f3Lbz8nbFWTwVHhzbzFz7d6CZO73aubR-?usp=drive_link
Bug report	https://docs.google.com/document/d/1rP2H4H34jCnAHYyww_E8ZVCZuIZSTIkY/edit?usp=sharing&oid=113345083993190881514&rtpof=true&sd=true

UI Automation Testing (Selenium + TestNG)

UI automation testing was implemented to validate critical user workflows and reduce regression risks after system changes. Selenium WebDriver integrated with TestNG was selected due to its stability, flexibility, and suitability for Java-based automation frameworks.

Automation efforts focused on high-risk and high-impact scenarios, including user authentication, product sorting, cart operations, checkout, logout, and invalid login handling, ensuring reliability across core application functionalities.

Test Scenarios Automated

- User logs in with valid credentials and successfully navigates to the Products page
- All available products are added to the cart from the products listing page
- Selected products are removed from the cart while validating cart updates
- Products are sorted using randomly selected sorting options (name and price)
- User navigates to the cart and initiates the checkout process
- Checkout information is entered using randomly generated user data (first name, last name, and postal code)
- The checkout process is completed, and a successful order confirmation message is validated
- User returns to the products page after completing the order
- Additional products are added to the cart, and items are removed from both the cart and the product details page
- User logs out successfully from the application

These scenarios represent a complete end-to-end user flow and ensure that critical functionalities such as authentication, product management, checkout, and logout behave correctly after system changes.

How to Run the Automation Tests

To execute the UI automation tests, follow these steps:

1. Install Java JDK and Eclipse IDE
2. Add Selenium WebDriver and TestNG dependencies to the project
3. Run the TestNG test classes directly from the IDE
4. Review execution results, logs, and assertions in the console

Automation Results Process

UI automation testing for SauceDemo was implemented using Selenium WebDriver with TestNG. Eight test cases were executed sequentially, covering the full end-to-end user journey.

The tests validated login, product sorting, cart operations, checkout with random user data, order completion, and logout. Set up and teardown ensured clean browser sessions, and screenshots were captured on failures.

Automation Run	https://drive.google.com/file/d/1SiEE8Q5pwmHTufyEHCMevwCWAOf_J8LF/view?usp=sharing
-----------------------	---

CI/CD and Version Control

Effective version control and automation were essential to ensure consistency, traceability, and smooth execution of the QA activities.

Git Workflow and Repository Structure

A structured Git workflow was used with a central repository containing clearly organized folders for test plans, manual test cases, Postman collections, automation scripts, and reports. This organization made it easy to locate files and maintain project clarity.

Branching and Commit Strategy

Branches were used for separate tasks such as manual testing, API testing, and UI automation. Commits were descriptive, reflecting the changes made, which allowed efficient tracking and collaboration.

Benefits of CI/CD Integration

Integrating automated testing within a CI/CD pipeline provides several benefits, including:

- Greater assurance in the reliability and stability of the application
- Enhanced test coverage using automated scripts
- Lower reliance on manual testing efforts

- Quicker feedback on updates and code changes
- Early identification of bugs and issues

Automation Execution via GitHub Actions

Automation scripts were integrated with GitHub Actions to allow consistent execution whenever changes were pushed. This provided an automated and repeatable process for running UI and API tests.

GitHub Repository	https://github.com/Rania-emad/Quality.git
--------------------------	---

Risks, Limitations, and Challenges

- Non-persistent data in DummyJSON
- Limited testing resources
- Time constraints for completing manual and automated tests
- Human errors during manual testing
- Performance test limitations due to laptop capacity

Conclusion

This QA project provided a comprehensive assessment of the Loyalty+ Store platform. Manual, API, performance, and UI automation testing were performed to ensure functional correctness, reliability, and responsiveness. Defects were identified and documented, automation scripts improved repeatability, and performance tests offered insights into potential bottlenecks. Overall, the testing confirmed that the system is stable for critical user flows, and the portfolio demonstrates a structured and professional QA approach.

Appendix

Swag Labs	https://www.saucedemo.com/
DummyJSON	https://dummyjson.com/docs#intro-ip
Eclipse IDE	https://eclipseide.org/
Postman	https://www.postman.com/
GitHub Repository	https://github.com/Rania-emad/Quality.git
Automation Run	https://drive.google.com/file/d/1SiEE8Q5pwmHTufyEHCMevwCWAOf_J8LF/view?usp=sharing
K6	https://drive.google.com/drive/folders/1dfVc4Jx_RU03-Mz0gJOxo4R9BTEkWdQr?usp=drive_link
Newman+ Html report	https://drive.google.com/drive/folders/1f3Lbz8nbfWTwVHhbfzFz7d6CZO73aubR-?usp=drive_link
Bug report	https://docs.google.com/document/d/1rP2H4H34jCnAHYyww_E8ZVCZuIZSTIkY/edit?usp=sharing&oid=113345083993190881514&rtpof=true&sd=true
Test plan	https://docs.google.com/document/d/1hnrHvD0MdYAzM0lw_qLEn04fwwdGmEAE/edit?usp=drive_link&oid=113345083993190881514&rtpof=true&sd=true
Testcases	https://docs.google.com/document/d/1ETYhPYnsdmlisSEZ5Xg3teMcy9b1Qj22/edit?usp=drive_link&oid=113345083993190881514&rtpof=true&sd=true